AI反馈报告样例（标准案例）

> **学生编号**：S2302-15（已匿名化）
> **作业批次**：第10次作业——MVC综合实战
> **作业主题**：实现"课程作业管理系统"的作业发布与查看功能（个性化任务：图书借阅管理）
> **工具**：Cursor IDE（内置Claude模型）
> **使用场景**：提交代码后，使用AI进行自审查并撰写反馈报告
> **提交时间**：2025年10月29日 21:47

一、学生提交的代码结构

```
src/main/java/com/tyust/library/
├── controller/
|   └── BookServlet.java          ← 问题集中区
├── dao/
|   └── BookDao.java              ← 存在SQL注入
├── model/
|   └── Book.java
└── (缺少service包)               ← 架构缺失

src/main/webapp/
├── WEB-INF/web.xml
├── book/
|   ├── list.jsp                    ← 含scriptlet代码
|   └── add.jsp
└── index.jsp
```

二、AI审查请求

学生输入的提示词

```
请审查我的Java Web MVC项目代码（图书借阅管理系统），重点检查以下方面：

1. MVC分层是否清晰（Controller/Service/DAO/Model）
2. Servlet中是否存在业务逻辑泄漏（应在Service层）
3. JSP中是否存在Java代码（应使用EL+JSTL）
4. 数据库操作是否正确使用PreparedStatement
5. 异常处理是否合理

请按以下格式输出每个问题：
- 文件与行号
- 问题描述
- 严重程度（高/中/低）
- 对应MVC知识点
- 修改建议
```

学生附上的代码文件

BookServlet.java（核心问题代码）：

```
@WebServlet("/book")
public class BookServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse
response)
            throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");
        String action = request.getParameter("action");

        if ("list".equals(action)) {
            // 问题1：Servlet中直接写SQL——业务逻辑泄漏到Controller
            Connection conn = null;
            try {
                conn = DBUtil.getConnection();
                String sql = "SELECT * FROM books ORDER BY id DESC";
                Statement stmt = conn.createStatement();
                ResultSet rs = stmt.executeQuery(sql);
                List<Book> books = new ArrayList<>();
                while (rs.next()) {
                    Book b = new Book();
                    b.setId(rs.getInt("id"));
                    b.setTitle(rs.getString("title"));
                    b.setAuthor(rs.getString("author"));
                    b.setStatus(rs.getString("status"));
                    books.add(b);
                }
                request.setAttribute("books", books);
                request.getRequestDispatcher("/book/list.jsp").forward(request,
response);
            } catch (Exception e) {
                // 问题4：空catch块——异常被吞掉
            } finally {
                if (conn != null) try { conn.close(); } catch (Exception e) {}
            }

        } else if ("search".equals(action)) {
            // 问题3：SQL字符串拼接——SQL注入风险
            String keyword = request.getParameter("keyword");
            BookDao dao = new BookDao();
            List<Book> results = dao.searchByTitle(keyword);
            request.setAttribute("books", results);
            request.getRequestDispatcher("/book/list.jsp").forward(request,
response);
        }
    }
}
```

BookDao.java（SQL注入问题）：

```
public class BookDao {
```

```
    public List<Book> searchByTitle(String title) {
        List<Book> list = new ArrayList<>();
        try {
            Connection conn = DBUtil.getConnection();
            // SQL注入风险：直接拼接用户输入
            String sql = "SELECT * FROM books WHERE title LIKE '%" + title +
"%'";
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql);
            while (rs.next()) {
                Book b = new Book();
                b.setId(rs.getInt("id"));
                b.setTitle(rs.getString("title"));
                b.setAuthor(rs.getString("author"));
                b.setStatus(rs.getString("status"));
                list.add(b);
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
        return list;
    }
}
```

list.jsp（scriptlet问题）：

```
<%@ page import="java.util.List, com.tyust.library.model.Book" %>
<html>
<body>
<h2>图书列表</h2>
<table border="1">
<%
    // 问题2：JSP中使用scriptlet——应使用EL+JSTL
    List<Book> books = (List<Book>) request.getAttribute("books");
    if (books != null) {
        for (Book b : books) {
%>
    <tr>
        <td><%= b.getId() %></td>
        <td><%= b.getTitle() %></td>
        <td><%= b.getAuthor() %></td>
        <td><%= b.getStatus() %></td>
    </tr>
<%
        }
    }
%>
</table>
</body>
</html>
```

三、AI审查输出（完整记录）

```
我审查了你的图书借阅管理系统代码，发现以下5个问题，按严重程度排序：

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
问题1（严重程度：高）—— Service层缺失
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

文件：项目结构（无BookService.java）
问题描述：项目中缺少Service层，BookServlet直接操作数据库连接
（第12-29行），将数据访问逻辑写在了Controller中。这违反了
MVC模式中Controller只负责"请求分发"的原则。
对应知识点：MVC模式、三层架构（Controller → Service → DAO）、
单一职责原则
修改建议：
1. 新建 com.tyust.library.service.BookService 类
2. 将BookServlet中的数据库操作迁移到BookService
3. BookService调用BookDao完成数据访问
4. BookServlet只负责：接收参数 → 调用Service → 设置属性 → 转发JSP

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
问题2（严重程度：高）—— SQL注入漏洞
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

文件：BookDao.java 第6行
问题描述：searchByTitle方法使用字符串拼接构造SQL查询：
  "SELECT * FROM books WHERE title LIKE '%" + title + "%'"
如果用户输入 title = "'; DROP TABLE books; --",
拼接后的SQL将变为：
  SELECT * FROM books WHERE title LIKE '%'; DROP TABLE books; --%'
这将导致books表被删除。
对应知识点：SQL注入（SQL Injection）、PreparedStatement参数化查询
修改建议：
  String sql = "SELECT * FROM books WHERE title LIKE ?";
  PreparedStatement pstmt = conn.prepareStatement(sql);
  pstmt.setString(1, "%" + title + "%");
  ResultSet rs = pstmt.executeQuery();

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
问题3（严重程度：中）—— JSP中使用scriptlet代码
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━

文件：list.jsp 第6-18行
问题描述：使用<% %>脚本代码和<%= %>表达式输出数据。这种写法
将Java逻辑嵌入视图层，违反了MVC的关注点分离原则，且难以维护。
对应知识点：EL表达式、JSTL标签库、JSP最佳实践
修改建议：
  <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
  <c:forEach var="book" items="${books}">
    <tr>
      <td>${book.id}</td>
      <td>${book.title}</td>
      <td>${book.author}</td>
      <td>${book.status}</td>
    </tr>
  </c:forEach>

━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
```

```
问题4（严重程度：中）—— 空catch块
━━━
```

#	AI建议	我的决策	判断理由	花了多久理解
4	catch块补充异常处理	✓ 采纳	空catch确实有问题。AI建议的"记录日志+跳转错误页"比我想的"打印堆栈"更合理。	约1分钟
5	添加分页功能	✗ 拒绝	本次作业要求是MVC基本CRUD，分页不在要求范围内。如果加分页还需要改JSP和DAO，工作量大但不得分。	立即判断拒绝

修改前后diff汇总

```
项目结构变化:
+ src/main/java/com/tyust/library/service/BookService.java  (新增)

BookServlet.java:
- Connection conn = null;
- try { conn = DBUtil.getConnection(); ... }  (删除28行数据库操作代码)
+ BookService bookService = new BookService();
+ List<Book> books = bookService.listAll();     (改为调用Service)

BookDao.java:
- String sql = "SELECT * FROM books WHERE title LIKE '%" + title + "%'";
- Statement stmt = conn.createStatement();
+ String sql = "SELECT * FROM books WHERE title LIKE ?";
+ PreparedStatement pstmt = conn.prepareStatement(sql);
+ pstmt.setString(1, "%" + title + "%");

list.jsp:
- <%@ page import="java.util.List, com.tyust.library.model.Book" %>
- <% List<Book> books = (List<Book>) request.getAttribute("books"); ...  (删除12行scriptlet)
+ <%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
+ <c:forEach var="book" items="${books}"> ...   (改为EL+JSTL)
```

反思（200字）

> AI帮我找到了5个问题，其中3个是我**明知道正确做法但写代码时忘记了**（SQL注入、EL表达式、三层架构）。这说明"知道知识点"和"写代码时能正确应用"之间有很大差距。传统的学习方式（听课→做题→考试）很难弥补这个差距，因为你总觉得自己会了，直到AI指出代码中的具体错误行号，才发现自己其实没做到。
>
> AI也给了一个不必要的建议（添加分页）。我拒绝了它，因为本次作业的目标是掌握MVC分层，不是做一个完整产品。如果不加判断地全部采纳，反而偏离了学习重点。
>
> **结论**：AI审查最大的价值是"把知识点对照到我自己的代码中"——这比看课本例子有效得多。但AI不了解作业要求和课程进度，所以必须自己判断哪些建议该采纳。

五、教师批注

> **AI使用规范性：★★★★★（5/5）**
> 提示词结构清晰，按任务卡格式输入，附上了完整代码。
>
> **批判性思维：★★★★☆（4/5）**
> 对5条建议逐条判断，拒绝理由合理。扣1分原因：对问题4的采纳可以更深入——AI建议的`System.err.println`在生产中也不够好，应考虑日志框架，但这超出本课程范围，不强求。
>
> **知识点对照：★★★★★（5/5）**
> 每条判断都明确对应了课堂讲次和知识点（"第9讲MVC""第6讲JDBC""第4讲EL"），说明学生在AI辅助下主动回顾了前序知识。
>
> **修改质量：★★★★★（5/5）**
> Service层补充完整，SQL注入修复正确，JSP改写规范。代码在修改后通过了全部8个功能测试点。
>
> **综合评价**：这是AI辅助自查的标准范例。学生展示了"AI发现→人工判断→动手修改→回顾知识"的完整闭环。特别值得肯定的是对分页建议的拒绝——能区分"正确但不必要"的建议是AI素养的重要表现。
>
> **成绩**：本次作业 9.2/10（功能8/8 + 规范1.2/2）
> ——教师评语，2025年11月1日